

PROPOSTA DE ESTRUTURA - PROJETO DE SA

Organizar um projeto usando **MVC (Model-View-Controller)** e **DAO (Data Access Object)** é uma **excelente** prática para manter o código limpo, dividido e fácil de dar manutenção.

Para ficar claro: o **Flask** vai centralizar o seu **Controller** (gerenciando as rotas) e também vai renderizar as suas **Views** (através de arquivos HTML/Jinja2). O **MySQL** será o destino final dos seus dados, gerenciado pela camada **DAO**.

Estrutura de Arquivos

Essa estrutura separa rigidamente as responsabilidades de cada componente:

Plaintext

```
meu_projeto/
├── DOCS/                # Pasta onde devem ser armazenados todos os documentos
├── config.py            # Configurações globais (credenciais do banco, chaves, etc.)
├── run.py               # Arquivo principal para iniciar a aplicação
├── requisitos.txt       # Dependências do projeto (flask, mysql-connector, etc.)
└── app/
    ├── __init__.py      # Inicializa o Flask e junta as peças
    ├── models/          # [MODEL] Classes que representam as tabelas do banco
    │   ├── __init__.py
    │   └── usuario.py
    ├── dao/              # [DAO] Isolamento das consultas SQL do MySQL
    │   ├── __init__.py
    │   ├── db_connection.py # Gerenciador de conexão com o MySQL
    │   └── usuario_dao.py  # Métodos CRUD específicos do usuário
    ├── controllers/      # [CONTROLLER] Rotas do Flask e lógica de requisição
    │   ├── __init__.py
    │   └── usuario_controller.py
    ├── templates/        # [VIEW] Páginas HTML que o usuário vai ver
    │   ├── base.html      # Layout padrão (Header, Navbar, Footer)
    │   └── usuario/
    │       ├── cadastro.html
    │       └── login.html
```

Agora vamos detalhar alguns arquivos da estrutura acima para que você e sua equipe entendam exatamente o papel de cada um no ecossistema do projeto. Para ajudar a visualizar como esses arquivos conversam entre si quando o usuário interage com o sistema, veja o fluxo abaixo:

Arquivos da Raiz do Projeto

config.py

- **Funcionalidade:** Centraliza variáveis de configuração globais, como as credenciais do banco de dados MySQL (host, usuário, senha, nome do banco) e chaves secretas do Flask.

- **Explicação:** Pensem nele como o **painel de controle** do projeto. Em vez de espalhar a senha do banco de dados por vários arquivos, você coloca tudo aqui. Se amanhã o banco mudar de endereço, você altera apenas esse arquivo.

run.py

- **Funcionalidade:** É o ponto de entrada da aplicação. Ele importa a configuração do servidor e inicia o projeto.
- **Explicação:** É a **chave de ignição** do motor. Quando você digita no terminal `python run.py`, é esse arquivo que "dá a partida" no Flask para o projeto ficar online.

requisitos.txt (ou requirements.txt)

- **Funcionalidade:** Uma lista de texto contendo todas as bibliotecas externas que o projeto precisa para rodar (ex: `Flask`, `mysql-connector-python`).
- **Explicação:** É a **lista de compras** do ambiente de desenvolvimento. Quando um colega de equipe baixa o projeto, ele roda um comando único (`pip install -r requisitos.txt`) e o Python instala tudo o que falta automaticamente.

DOCS/

- **Funcionalidade:** Esta pasta serve para vocês armazenarem todos os documentos (não código) relevantes do projeto, como por exemplo: Script de criação do Banco de Dados, Arquivos de Modelagem, Documentos entre outros.

Pasta app/ (O Coração da Aplicação)

app/__init__.py

- **Funcionalidade:** Transforma a pasta `app` em um pacote Python, inicializa o objeto do Flask e registra as rotas (Blueprints).
- **Explicação:** Ele funciona como o **gerente de integração**. Assim que o projeto é iniciado pelo `run.py`, o `__init__.py` acorda, cria o aplicativo Flask, lê as configurações e junta os Controllers e as Views para trabalharem em harmonia.

Camada Model (Regras de Negócio e Dados)

app/models/usuario.py

- **Funcionalidade:** Contém a classe `Usuario` escrita em Python puro, representando a estrutura da tabela do banco de dados (atributos como `id`, `nome`, `email`).
- **Explicação:** É o **molde ou fôrma** do dado. Ele não sabe o que é HTML e não sabe o que é SQL; ele apenas sabe como um Usuário deve ser estruturado dentro da memória do computador.

Camada DAO (Data Access Object)

app/dao/db_connection.py

- **Funcionalidade:** Abre e fecha conexões com o banco de dados MySQL usando as credenciais do `config.py`.
- **Explicação:** É a **ponte levadiça**. Sempre que o sistema precisar falar com o MySQL, ele pede para este arquivo abrir um canal seguro de comunicação e, após o uso, fechar esse canal para não desperdiçar memória.

app/dao/usuario_dao.py

- **Funcionalidade:** Contém a classe `UsuarioDAO`, onde ficam armazenadas as funções que executam comandos SQL diretamente (como `INSERT`, `SELECT`, `UPDATE`, `DELETE`).

Explicação: É o **tradutor/mensageiro**. É o único arquivo do sistema que "fala SQL". Quando o sistema quer cadastrar alguém, o Controller entrega um objeto `Usuario` (do Model) para o DAO, e o DAO traduz isso em um comando `INSERT INTO usuario...` para o MySQL.

Camada Controller (A Lógica e as Rotas)

`app/controllers/usuario_controller.py`

- **Funcionalidade:** Define as URLs/rotas do site (ex: `/cadastro`, `/login`), recebe os dados enviados pelos formulários e decide o que fazer.

- **Explicação:** É o **guarda de trânsito ou recepcionista**. Quando o usuário clica em "Enviar" no formulário de cadastro, a requisição bate aqui. O Controller pega os dados do formulário, valida se estão certos, chama o DAO para salvar no banco e decide: *"Ok, agora vou mandar o usuário para a página de sucesso"*.

Camada View (A Interface do Usuário)

`app/templates/base.html`

- **Funcionalidade:** Contém a estrutura HTML fixa que se repete em todas as páginas (como a barra de navegação superior, o rodapé e as importações do CSS/Bootstrap).

- **Explicação:** É o **esqueleto visual**. Para você não ter que copiar e colar o menu do site em todas as páginas do projeto, você faz isso uma vez só aqui. As outras páginas apenas "encaixam" o conteúdo delas dentro desse esqueleto.

`app/templates/usuario/cadastro.html` e `login.html`

- **Funcionalidade:** Páginas HTML específicas que contêm os formulários de entrada de dados utilizando a sintaxe do Jinja2 para se integrarem ao Flask.

- **Explicação:** É a **vitrine**. É o que o usuário final enxerga na tela do navegador. O formulário envia os dados através do atributo `method="POST"` direto para o Controller processar.

Roteiro do Projeto: Passo a Passo

Entender o roteiro de execução é fundamental para que o projeto não saia dos trilhos. Abaixo, apresento um detalhamento aprofundado do **"Roteiro do Projeto"**, expandindo as instruções com práticas de mercado e passos práticos para facilitar a implementação da sua equipe:

Etapa 1: Configuração do Ambiente e Banco de Dados

Esta é a fundação do projeto. Sem um ambiente isolado e um banco estruturado, o código não terá onde rodar ou armazenar informações.

- **Ambiente Python:** O primeiro passo é criar um ambiente virtual (venv) para isolar as bibliotecas deste projeto das demais no seu computador e, em seguida, instalar o Flask e o conector do MySQL (sendo recomendados o `mysql-connector-python` ou `pymysql`).

Aprofundamento: No terminal, você usará comandos como `python -m venv venv` para criar o ambiente e ativá-lo. Depois, ao instalar as dependências com `pip install`, lembre-se de salvar a lista exata rodando `pip freeze > requisitos.txt`. Isso garante que sua equipe tenha a "lista de compras" correta.

- **Modelagem SQL:** No MySQL, é necessário criar o banco de dados e as tabelas correspondentes, como a tabela `usuario`. Vocês devem guardar o script `.sql` resultante na pasta do projeto.

Aprofundamento: Crie um arquivo chamado `init_db.sql` na raiz do projeto. Nele, escreva os comandos `CREATE DATABASE` e `CREATE TABLE`. Isso serve como documentação e permite que qualquer novo membro da equipe recrie o banco de dados idêntico ao seu em questão de segundos.

Etapa 2: A Conexão e a Camada DAO (Data Access Object)

Aqui construímos as "pontes" e os "veículos" que transportam a informação do código para o banco de dados.

- **A Ponte de Conexão:** No arquivo `db_connection.py`, criem uma classe ou função que abre e fecha a conexão com o MySQL usando os dados do `config.py`.

Aprofundamento: É vital garantir que a conexão seja fechada (usando `cursor.close()` e `connection.close()`) mesmo que ocorra um erro durante a consulta. O uso de blocos `try/except/finally` em Python é a melhor prática para isso.

- **O Veículo (Model):** No `models/usuario.py`, criem a classe `Usuario` contendo os atributos do banco (id, nome, email, senha) e o método construtor `__init__`. Essa classe servirá exclusivamente para transportar os dados pelo sistema.

- **O Mensageiro (DAO):** No `dao/usuario_dao.py`, criem a classe `UsuarioDAO` para abrigar funções com comandos SQL puros, como `inserir_usuario(usuario)` ou `buscar_por_id(id)`.

Aprofundamento: Nunca concatene variáveis diretamente na string SQL (ex: `f"SELECT * FROM usuario WHERE nome = {nome}"`). Use *queries parametrizadas* (ex: `WHERE nome = %s`) fornecidas pelo conector para evitar ataques de SQL Injection.

Etapa 3: A Camada Controller (Rotas Flask)

O Controller é o cérebro das operações web. Ele recebe o tráfego de usuários e decide como orquestrar o Model e o DAO.

- **Organização de Rotas:** No arquivo `controllers/usuario_controller.py`, criem as rotas utilizando Blueprints, que é a maneira do Flask separar rotas em diferentes arquivos.

- **Fluxo de Requisição:** Em rotas de submissão (como `@usuario.route('/cadastrar', methods=['POST'])`), o papel de vocês é capturar os dados do formulário HTML, criar uma instância do Model `Usuario`, enviar para o DAO salvar e redirecionar o usuário.

Aprofundamento: Antes de enviar os dados para o DAO, o Controller deve **validar** as informações. Verifique se os campos não estão vazios, se o e-mail tem um formato válido ou se a senha atende aos requisitos mínimos. Se houver erro, devolva o usuário para a página de cadastro com uma mensagem de aviso em vez de travar o sistema.

Etapa 4: A Camada View (Interface)

É aqui que o "esqueleto" visual ganha vida para o usuário final.

- **Layout Base:** No arquivo `templates/base.html`, estruturem o HTML5 principal e adicionem o CSS ou Bootstrap para dar estilo.

Aprofundamento: Usem as tags do Jinja2 como `{% block content %}{% endblock %}` neste arquivo base. É nesse espaço reservado que o conteúdo das outras páginas será injetado.

- **Páginas Específicas:** Na pasta `templates/usuario/`, criem as páginas que herdarão o layout do `base.html` via Jinja2. O mais importante aqui é garantir que os atributos `name` dos formulários (ex: `<input name="email">`) sejam idênticos ao que o Controller está esperando receber.

Etapa 5: Junta Tudo e Roda

A amarração final para colocar o sistema no ar.

- **Registro no App:** Abram o arquivo `app/__init__.py` para configurar a criação da aplicação Flask e para registrar os Blueprints que vocês criaram nos controllers.

- **A Chave de Ignição:** No arquivo `run.py`, façam a importação do app e rodem o comando `app.run(debug=True)`.

Aprofundamento: O parâmetro `debug=True` é excelente para o desenvolvimento, pois ele reinicia o servidor automaticamente sempre que você salva um arquivo e mostra os erros detalhados no navegador. Lembre-se de remover ou alterar para `False` quando o sistema for publicado em um servidor real por questões de segurança.

Conclusão: O Objetivo Desta Proposta Estrutural

1. Padronização e Divisão Rígida de Responsabilidades

Este documento visa erradicar o código confuso (comumente chamado de "código espaguete") ao introduzir a combinação do padrão **MVC (Model-View-Controller)** com a camada **DAO (Data Access Object)**.

Ao separar rigidamente onde o dado é moldado (`Model`), onde o SQL é isolado (`DAO`), onde as rotas e regras de negócio transitam (`Controller`) e onde a interface é renderizada (`View`), este guia garante que cada membro da equipe saiba exatamente onde criar ou modificar uma funcionalidade.

2. Autonomia, Alinhamento e Continuidade da Equipe

Projetos em equipe frequentemente sofrem com gargalos de comunicação ou dependência de um único desenvolvedor. O objetivo do documento é democratizar o conhecimento técnico do ecossistema do projeto.

Detalhando os arquivos como o `config.py` (painel de controle), `requisitos.txt` (lista de compras de bibliotecas) e a pasta `DOCS/`, o guia cria um ambiente onde **qualquer integrante pode clonar o repositório, instalar as dependências e continuar o trabalho** sem atritos ou perda de tempo.

3. Roteirização Progressiva (Mitigação de Riscos)

O desenvolvimento de software pode ser esmagador se abordado de uma vez só. A divisão em **5 etapas consecutivas** cumpre o papel de estabelecer metas claras e cronológicas:

- **Fundação Segura:** Isolar o ambiente (`venv`) e documentar o banco (`init_db.sql`).
 - **Segurança e Conexão:** Blindar o sistema contra falhas de memória e ataques externos (como *SQL Injection* via queries parametrizadas).
 - **Orquestração e UX:** Validar dados no servidor antes de salvá-los e reaproveitar estruturas visuais com Jinja2 para agilizar o front-end.
 - **Acabamento:** Ativar o modo de depuração (`debug=True`) de forma consciente durante a construção.
-